Object Oriented Programming

CS22301

UNIT V

JAVAFX



A. Bamila Virgin Louis

Asso.Prof/ CSE

SXCCE

UNIT V
JAVAFX EVENT HANDLING, CONTROLS AND COMPONENTS

Course Outcome:

CO5: Design interactive GUI based applications using the concepts of event handling and JavaFX components.

UNIT V JAVAFX EVENT HANDLING, CONTROLS AND COMPONENTS

JAVAFX Events and Controls: Event Basics – Handling Key and Mouse Events. Controls: Checkbox, ToggleButton – RadioButtons – ListView – ComboBox – ChoiceBox – Text Controls – ScrollPane. Layouts – FlowPane – HBox and VBox – BorderPane – StackPane – GridPane. Menus – Basics – Menu – Menu bars – MenuItem.

JavaFX - INTRODUCTION

- JavaFX stands for **special effects in the Java language**.
- JavaFX is a collection of classes and interfaces that defines Java's modern graphical user interface (GUI).
- JavaFX supplies a diverse set of controls, such as buttons, scroll panes, text fields, check boxes, trees, and tables, that can be tailored to fit nearly any application.

JavaFX - INTRODUCTION

- Furthermore, effects, transforms, and animation can be employed to enhance the visual appeal of the controls.
- Thus, JavaFX not only enables you to build more exciting, visually appealing user interfaces, it also makes your job easier in the process.
- Simply put: JavaFX is a powerful, state-of-the-art GUI framework that is defining the future of GUI programming in Java.

The JavaFX Packages

- The JavaFX framework is contained in packages that begin with the **javafx** prefix.
- There are more than 30 JavaFX packages in its API library.
- Here are four examples:
- **javafx.application, javafx.stage, javafx.scene**, and **javafx.scene.layout**.

Setting the Stage with the Stage and Scene Classes

- The central metaphor implemented by JavaFX is the *stage*
- A stage contains a *scene*.
- A stage defines a space and a scene defines what goes in that space. Or, put another way, a stage is a container for scenes and a scene is a container for the items that comprise the scene.
- All JavaFX applications have at least one stage and one scene.

Setting the Stage with the Stage and Scene Classes

- **Stage** is a top-level container.
- All JavaFX applications automatically have access to one **Stage**, called the *primary stage*.
- The primary stage is supplied by the run-time system when a JavaFX application is started.
- Although you can create other stages, for many applications, the primary stage will be the only one required.
- **Scene** is a container for the items that comprise the scene.
- These can consist of various types of GUI elements, such as controls, text, and graphics.
- To create a scene, you will add elements to an instance of **Scene**. Then, set that **Scene** on a **Stage**.

Nodes and Scene Graphs

- The elements of a scene are called *nodes*. For example, a push button control is a node.
- However, nodes can also consist of groups of nodes. Furthermore, a node can have a child node.
- In this case, a node with a child is called a ***parent node* or *branch node***. Nodes without children are terminal nodes and are called ***leaves*.**
- The collection of all nodes in a scene creates what is referred to as a *scene graph*, which comprises a *tree*.

Nodes and Scene Graphs

- There is one special type of node in the scene graph, called the ***root node*.**
- This is the top-level node and is the only node in the scene graph tree that does not have a parent.
- Thus, with the exception of the root node, all other nodes have parents, and all nodes either directly or indirectly descend from the root node.
- The base class for all nodes is **Node**.
- There are several other classes that are, either directly or indirectly, subclasses of **Node**. These include **Parent, Group, Region**, and **Control**, to name a few.

Layouts

- JavaFX provides several layout panes that manage the process of **placing elements in a scene.**
- For example, the **FlowPane** class provides a flow layout and the **GridPane** class supports a row/column grid-based layout.
- Several other layouts, such as **BorderPane**, which organizes output within four border areas and a center, are available.
- Each inherits **Node**. The layouts are packaged in **javafx.scene.layout.**

The Application Class and the Life-Cycle Methods

- A JavaFX application must be a subclass of the **Application** class, which is packaged in **javafx.application**.
- Thus, your application class will extend **Application**.
- The **Application** class defines three life-cycle methods that your application can override. These are called **init( ), start( )**, and **stop( )**
  - void init( )
  - abstract void start(Stage *primaryStage*)
  - void stop( )

The Application Class and the Life-Cycle Methods

- The **init( )** method is called when the application begins execution.
- It is used to perform various initializations.
- As will be explained, it *cannot*, however, be used to create a stage or build a scene.
- If no initializations are required, this method need not be overridden because an empty, default version is provided.

The Application Class and the Life-Cycle Methods

- The **start( )** method is called after **init( )**.
- This is where your application begins, and it *can* be used to construct and set the scene.
- Notice that it is passed a reference to a **Stage** object.
- This is the stage provided by the run-time system and is the primary stage.
- Also notice that this method is abstract.
- Thus, it must be overridden by your application.
- When your application is terminated, the **stop( )** method is called.

Launching a JavaFX Application

- In general, when a JavaFX application begins execution, an instance of the subclass of **Application** defined by the application is created.
- Then **init( )**, followed by **start( )**, is executed. However, sometimes, such as in the case of a free-standing, self-contained JavaFX application, a call to the **launch( )** method defined by **Application** may be needed
- public static void launch(String … *args*)

Launching a JavaFX Application

- public static void launch(String … *args*)
- Here, *args* is a possibly empty list of strings that typically specifies command-line arguments.
- When called, **launch( )** causes the application to be constructed, followed by calls to **init( )** and **start( )**.
- The **launch( )** method will not return until after the application has terminated.
- This version of **launch( )** starts the subclass of **Application** from which **launch( )** is called.
- The second form of **launch( )** lets you specify a class other than the enclosing class to start.
- As a general rule, **launch( )** is called from **main( )**.

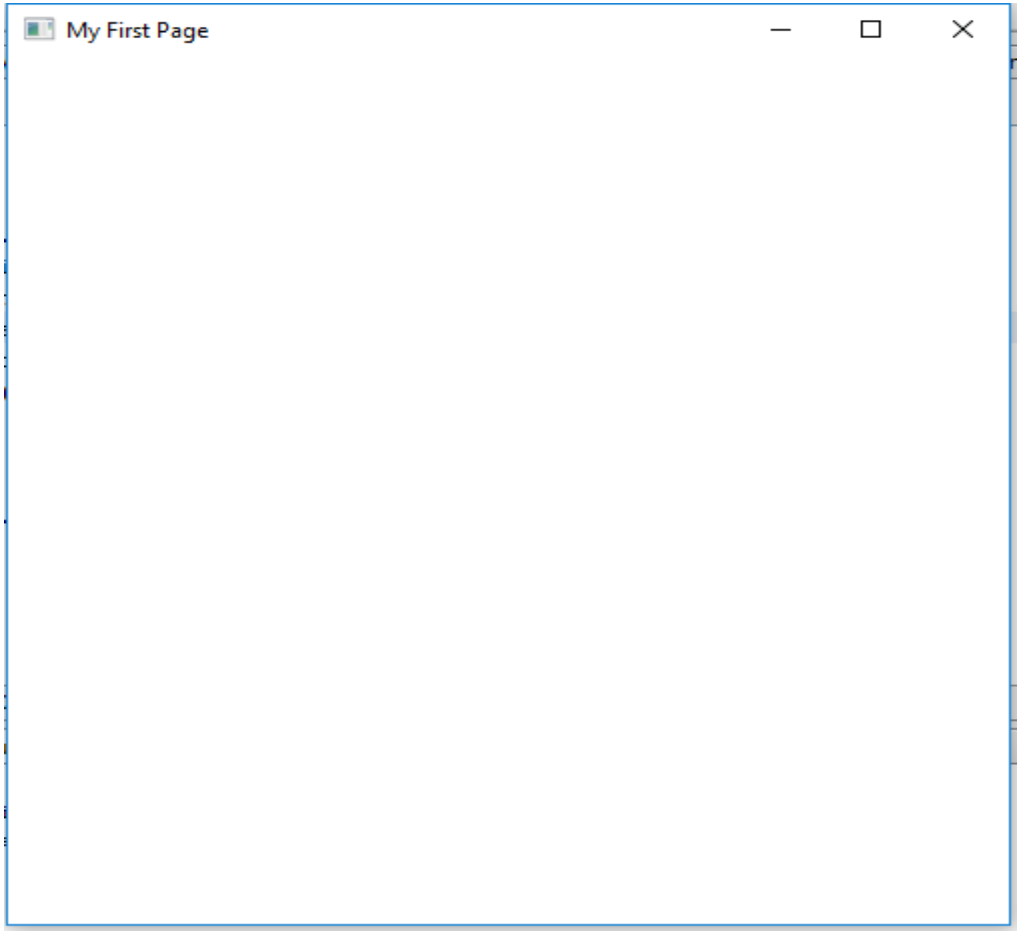
A JavaFX Application Skeleton


```
import javafx.application.*;
import javafx.scene.*;
import javafx.stage.*;
import javafx.scene.layout.*;
public class FirstProgram extends Application
{
     public static void main(String[] args)
   {
       launch(args);
   }
```

A JavaFX Application Skeleton

```
    public void start(Stage s)
    {
    System.out.println("inside start
method");
    s.setTitle("My First Page");
    FlowPane f=new FlowPane();
    Scene s1=new Scene(f,200,300);
    s.setScene(s1);
    s.show();
    }
}
```

Output



Build a Simple Scene Graph

- The label is one of the controls provided by JavaFX
- In JavaFX, a label is an instance of the **Label** class, which is packaged in **javafx.scene.control.**
- **Label** inherits **Labeled** and **Control**, among other classes.
- The **Labeled** class defines several features that are common to all labeled elements (that is, those that can contain text), and **Control** defines features related to all controls.
- The **Label** constructor that we will use is shown here:

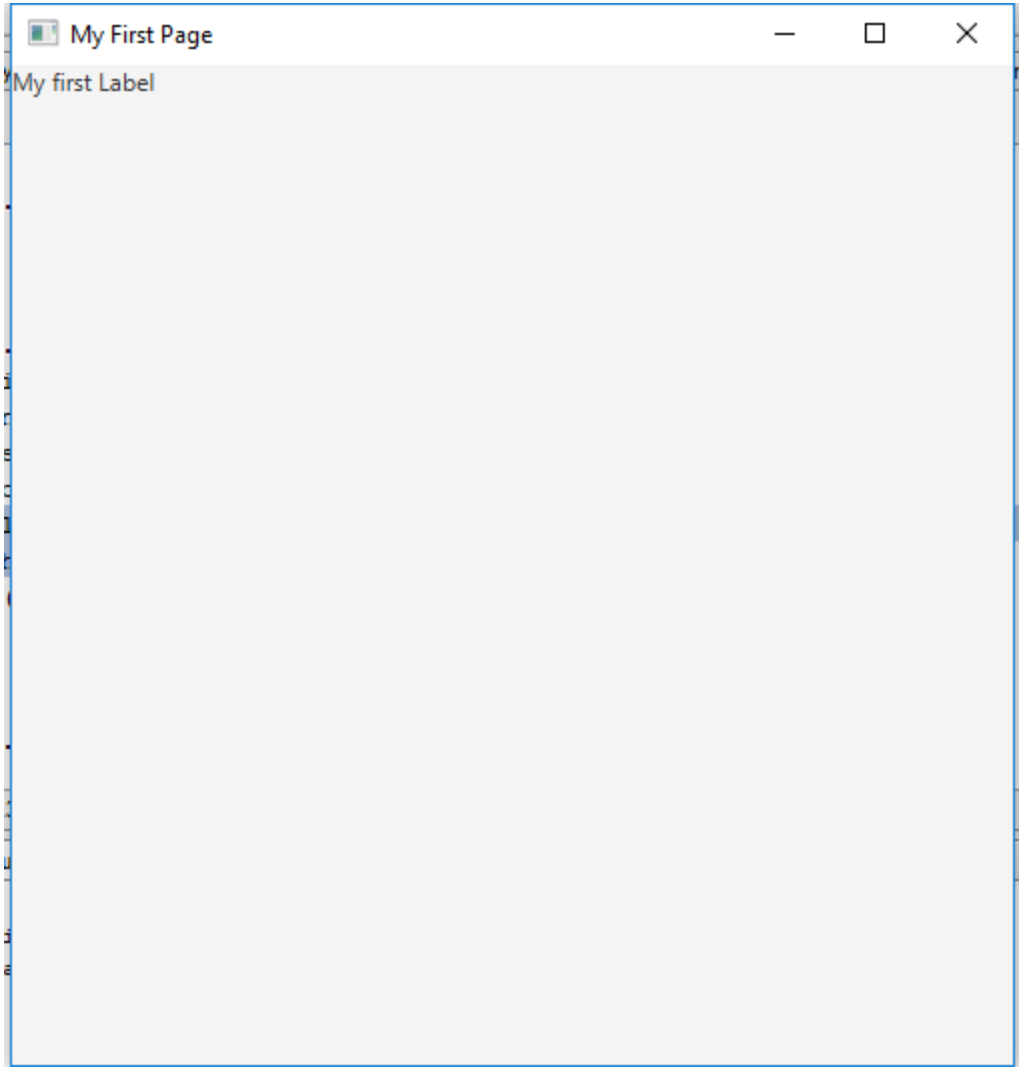
  Label(String *str*)
- The string that is displayed is specified by *str*.

INCLUDING a label


```
import javafx.application.*;
import javafx.scene.*;
import javafx.stage.*;
import javafx.scene.layout.*;
import javafx.scene.control.*;
public class FirstProgram extends Application
{
     public static void main(String[] args)
   {
      launch(args);
   }
```

A JavaFX Application Skeleton

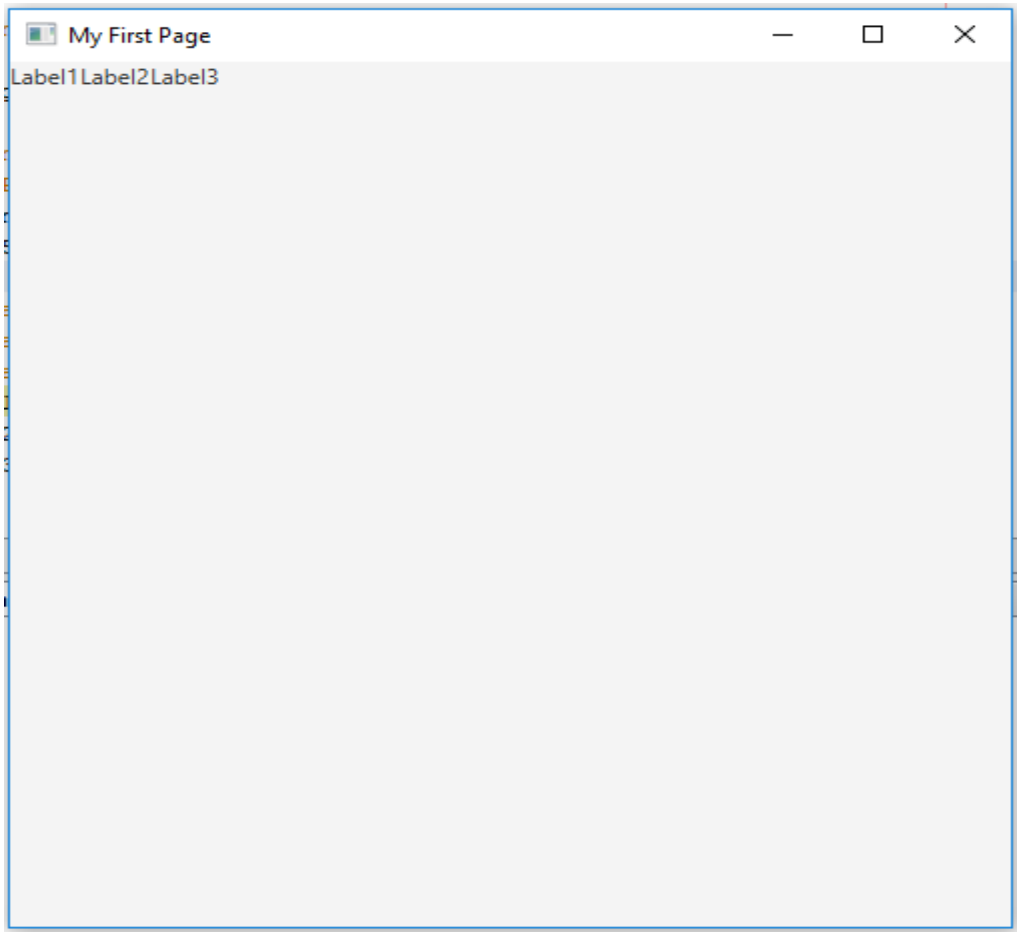
```
public void start(Stage s)
{
System.out.println("inside start method");
s.setTitle("My First Page");
FlowPane f=new FlowPane();
Scene s1=new Scene(f,200,300);
s.setScene(s1);
Label l=new Label("My first Label");
f.getChildren().add(l);
s.show();
}
}
```


My First Page
My first Label

MULTIPLE LABELS

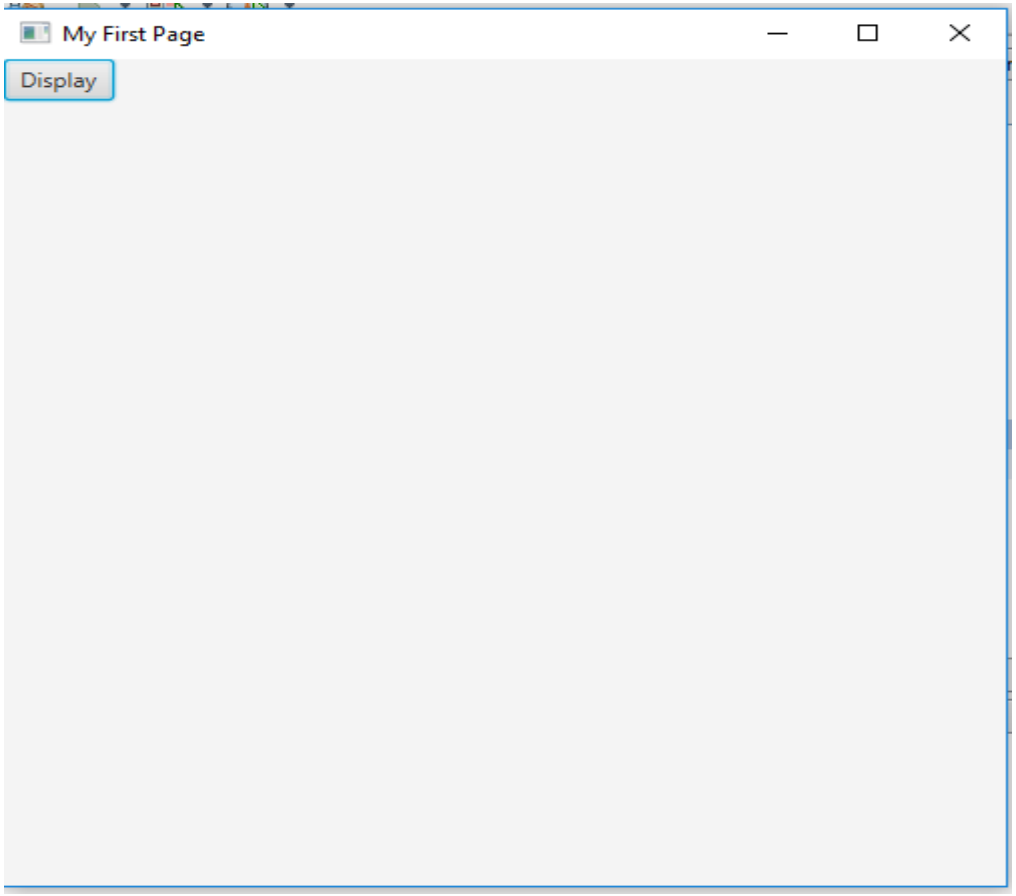

```
Label l1=new Label("Label1");
Label l2=new Label("Label2");
Label l3=new Label("Label3");
f.getChildren().add(l1);
f.getChildren().add(l2);
f.getChildren().add(l3);
```

MULTIPLE LABELS



TO ADD A BUTTON

```
Button b=new Button("Display");
f.getChildren().add(b);
```



CheckBox

- JavaFX defines a rich set of controls that are packaged in **javafx.scene.control**
- The immediate superclass of **CheckBox** is **ButtonBase**
- **CheckBox** supports three states.
  - checked
  - unchecked(default)
  - *indeterminate* (also called *undefined*).

CheckBox

- **CheckBox** constructors:
- CheckBox()
- CheckBox(String *str*)
- Methods:
- final boolean isSelected( ) – returns the state
- **setOnAction( )** - set the action event handler

Example

```
import javafx.application.*;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
public class Test extends Application
{
public void start(Stage s)
        {
        s.setTitle("My First Page");
        FlowPane f=new FlowPane();
        Scene s1=new Scene(f,200,300);
        CheckBox c=new CheckBox("First");
```

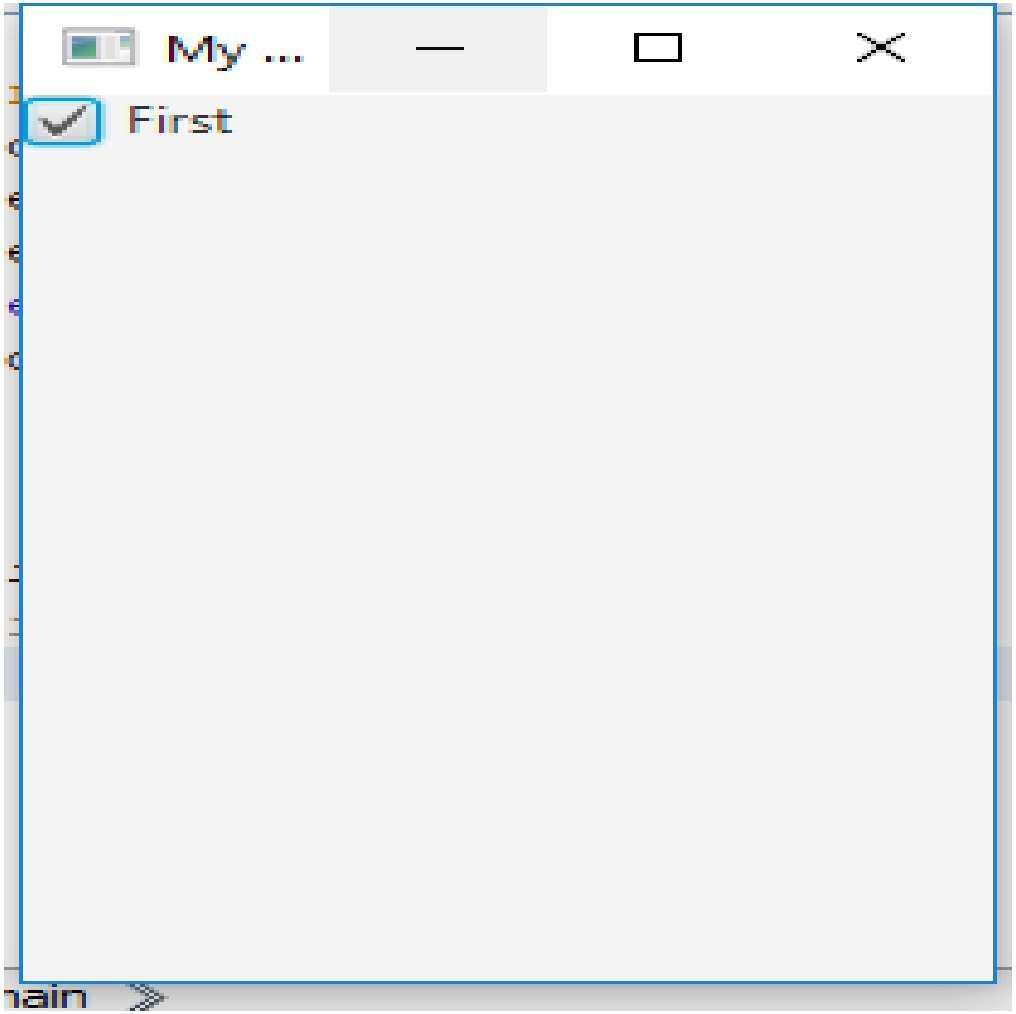
Example

```
        c.setSelected(true);
      f.getChildren().add(c);
      s.setScene(s1);
        s.show();
      }
   public static void main(String[] args)
    {
        launch(args);

   }

}
```

Output



ToggleButton

- A toggle button looks just like a push button, but it acts differently because it has two states: **pressed and released.**
- That is, when you press a toggle button, it stays pressed rather than popping back up as a regular push button does.
- When you click the toggle button a second time, it releases (pops up).
- Like **Button, ToggleButton** is also derived from **ButtonBase**

ToggleButton

- **Constructor:**
  - ToggleButton(String *str*)
- Methods:
  - final boolean isSelected( )

Example

```
import javafx.application.*;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
public class JavaApplication15 extends Application
{
public void start(Stage s)
        {
        s.setTitle("My First Page");
        FlowPane f=new FlowPane();
        Scene s1=new Scene(f,200,300);
        ToggleButton t=new ToggleButton("Button");
```

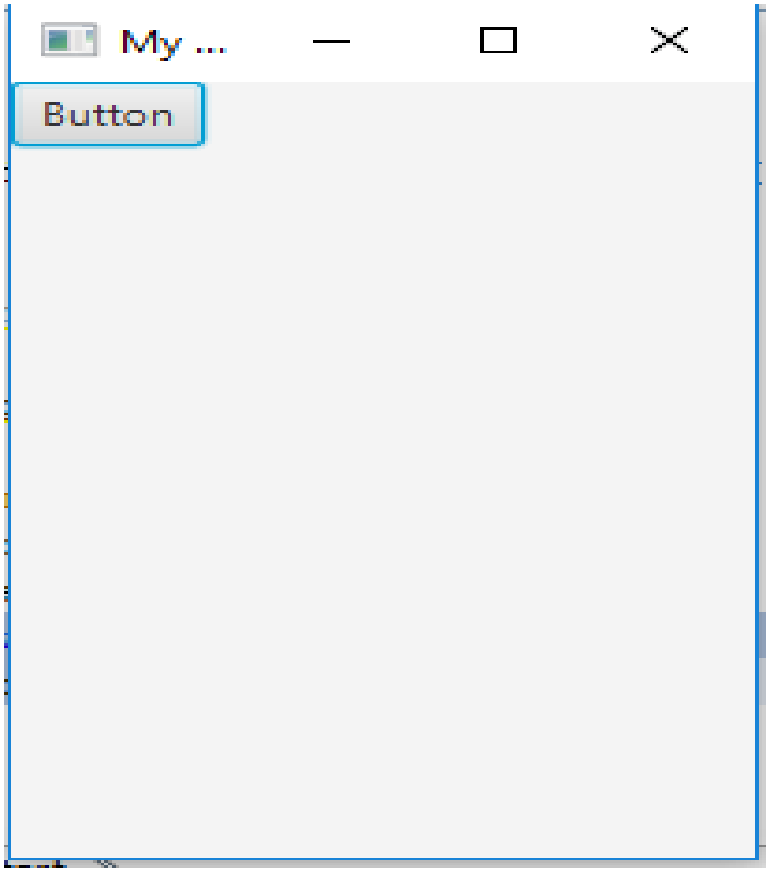
Example

```
        f.getChildren().add(t);
      s.setScene(s1);
        s.show();
      }
   public static void main(String[] args)
    {
        launch(args);

   }

}
```

Output



Text Controls

- JavaFX provides controls that let you enter text.
- JavaFX supports three forms of text controls.
- **TextField**, which allows one line of text to be entered;
- **TextArea**, which supports multiline text;
- **PasswordField**, which can be used to input passwords because what is typed is not shown.
- All three controls inherit **TextInputControl**, which defines much of the text control's functionality.

TextField

- **TextField** defines two constructors.
- The first is the default constructor, which creates an empty text field that has the default size.
- The second lets you specify the initial contents of the field.
- To specify a **TextField**'s size.
  - final void setPrefColumnCount(int *columns*)
- To get and set the text in a text field
  - final void setText(String *str*)
  - final string getText( )

TextField

- the keyboard commands for cut, copy, and paste are automatically supported by **TextField**
- You can also perform a number of operations under program control, such as deleting and inserting text.
- You can clear the content of a text field by calling **clear( )**.
- To set a prompting message that is displayed inside the text field when the user attempts to use a blank field.
  - final void setPromptText(String *str*)
- When the user presses ENTER while inside a **TextField**, an action event is generated.

PasswordField

- The **PasswordField** control works like **TextField**, except that the characters are not shown when they are typed by the user.

TextArea

- **TextArea** is also used much like **TextField**, except it does not generate an action event when ENTER is pressed
- A **TextArea** will have a default size, but you can set the size explicitly by using the **setPrefColumnCount()** and **setPrefRowCount( )** methods
- If the text inside a **TextArea** exceeds its width or height, scroll bars are added as needed.

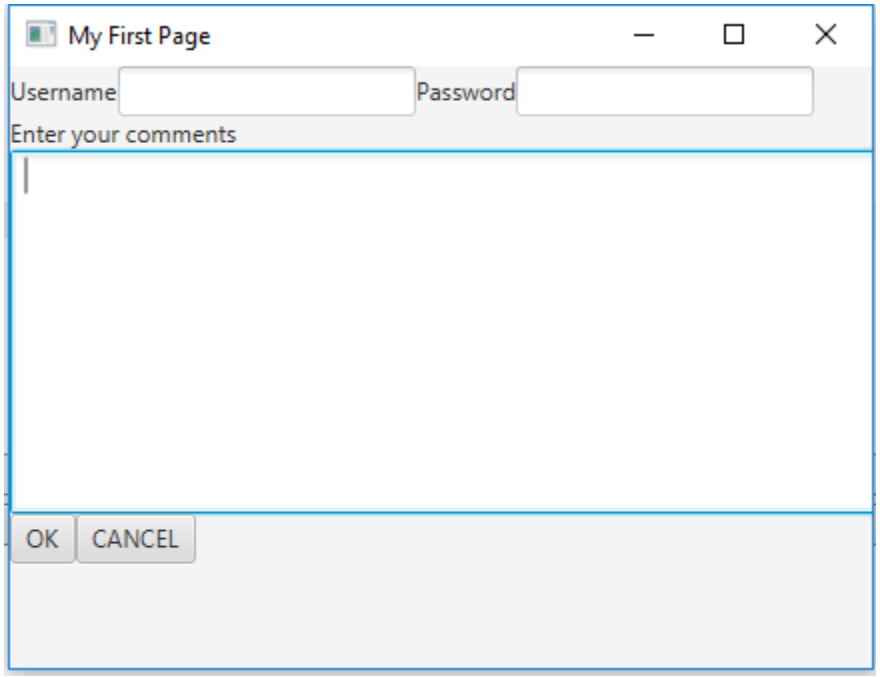
Example

```
import javafx.application.*;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
public class JavaApplication15 extends
Application{
public void start(Stage s)
{
```

```
s.setTitle("My First Page");
        FlowPane f=new FlowPane();
        Scene s1=new Scene(f,200,300);
      Label l1=new Label("Username");
      TextField t1=new TextField();
      Label l2=new Label("Password");
      TextField t2=new TextField();
       Label l3=new Label("Enter your comments");
       TextArea t3=new TextArea();
       Button b1=new Button("OK");
        Button b2=new Button("CANCEL");
```

```
 f.getChildren().addAll(l1,t1,l2,t2,l3,t3,b1,b2);
s.setScene(s1);
        s.show();
        }
    public static void main(String[] args)
 {
launch(args);
}
}
```

Example


My First Page
Username
Password
Enter your comments
OK
CANCEL

Radio Buttons

- Radio buttons are typically used to manage a group of mutually exclusive buttons.
- In such a group, only one button can be selected at any one time. They are supported by the **RadioButton** class, which extends both **ButtonBase** and **ToggleButton.**
- Radio buttons are the primary control employed when the user must select only one option among several alternatives.
- **Constructors:**
  - RadioButton(String *str*)

Radio Buttons

- Radio buttons are added to the toggle group by calling the **setToggleGroup( )** method on the button.
  - final void setToggleGroup(ToggleGroup *tg*)
- final void setSelected(boolean *state*)

Radio Buttons

- Radio buttons are added to the toggle group by calling the **setToggleGroup( )** method on the button.
  - final void setToggleGroup(ToggleGroup *tg*)
- final void setSelected(boolean *state*)

Example

```
import javafx.application.*;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
public class JavaApplication15 extends Application
{
public void start(Stage s)
        {
        s.setTitle("My First Page");
        FlowPane f=new FlowPane();
        Scene s1=new Scene(f,200,300);
        RadioButton b1=new RadioButton("One");
      RadioButton b2=new RadioButton("two");
```

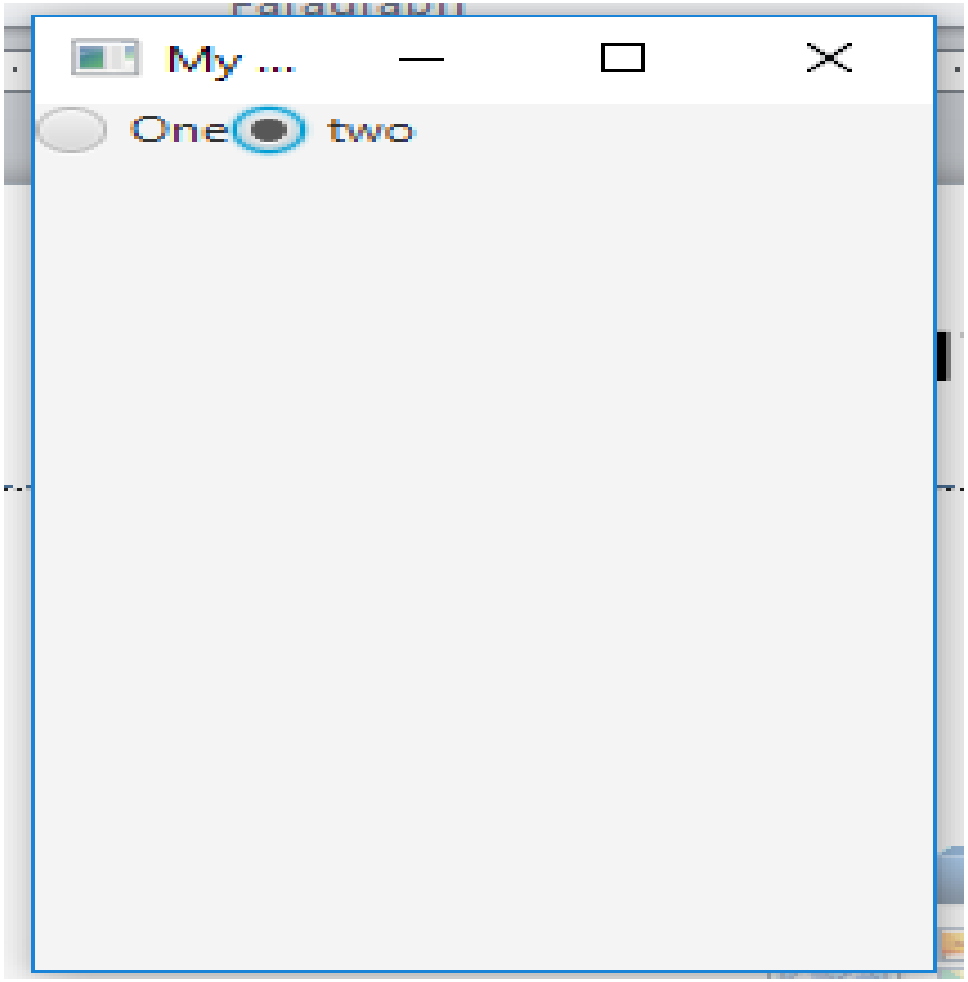
Example

```
    ToggleGroup t=new ToggleGroup();
     b1.setToggleGroup(t);
     b2.setToggleGroup(t);
     f.getChildren().addAll(b1,b2);
     s.setScene(s1);
          s.show();
     }
   public static void main(String[] args)
    {
          launch(args);

   }

}
```

Output



ListView

- *List views* are controls that display a list of entries from which you can select one or more.
- Furthermore, the number of entries in the list can increase or decrease during the execution of the program
- **ListView** is a generic class:

  class ListView<T>

- Here, **T** specifies the type of entries stored in the list view.

ListView

- **ListView** defines two constructors.
  - The first is the default constructor, which creates an empty **ListView**.
  - The second lets you specify the list of entries in the list.
    - ListView(ObservableList<T> *list*)
    - Here, *list* specifies a list of the items that will be displayed
    - It is an object of type **ObservableList**, which defines a list of observable objects.

ListView

- **Methods:**
  - final void setPrefHeight(double *height*)
  - final void setPrefWidth(double *width*)
  - void setPrefSize(double *width*, double *height*)
- **ListView** will automatically provide scroll bars when the items in the list exceed the size of the control.
- By default, a **ListView** allows only one item in the list to be selected at any one time.
- However, you can allow multiple selections by changing the selection mode

ListView

- **Enabling Multiple Selections:**
  - MultipleSelectionModel<String> m=l1.getSelectionModel();
- final void setSelectionMode(SelectionMode *mode*)
- Mode
  - **SelectionMode.MULTIPLE**
  - **SelectionMode.SINGLE.**

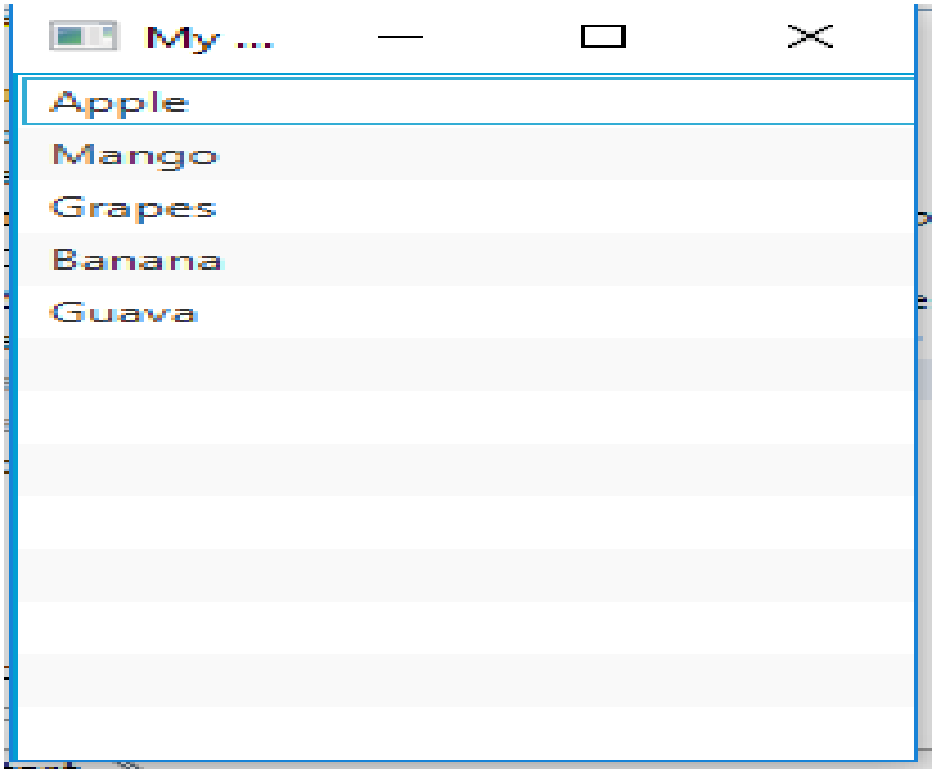
Example

```
import javafx.application.*;
import javafx.collections.*;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
```

```
public class Test extends Application{
public void start(Stage s)
        {
        s.setTitle("My First Page");
        FlowPane f=new FlowPane();
        Scene s1=new Scene(f,200,300);
    ObservableList<String> list=
FXCollections.observableArrayList("Apple","Mango","Grapes","
Banana","Guava");
        ListView<String> l1=new ListView(list);
    MultipleSelectionModel<String> m=l1.getSelectionModel();
    m.setSelectionMode(SelectionMode.MULTIPLE);
    f.getChildren().addAll(l1);
    s.setScene(s1);
        s.show();
    }
```

Example

```
public static void main(String[] args) {
launch(args);
    }
}
```

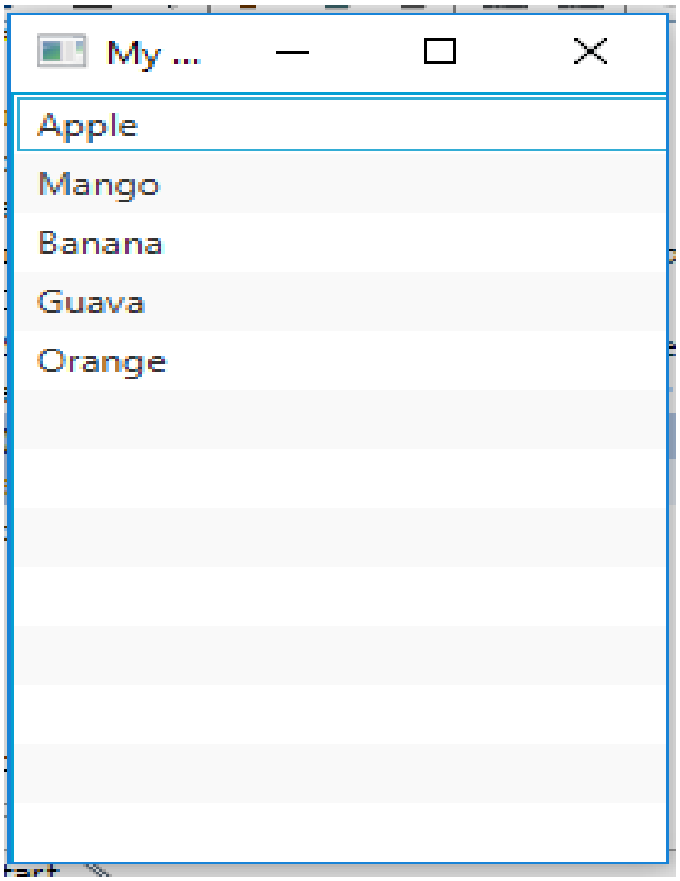


Changing the ListView Dynamically

- Once a list has been created, you can add to or remove items from it by adding or removing them from the **ObservableList** that backs it.
- To add an item, use **add( )**.
- To add a list of items, use **addAll()**.
- To remove an item, use **remove( )**.

EXAMPLE

```
list.add("Orange");
list.remove("Grapes");
```



ComboBoxes

- A combo box displays one selection, but it will also display a drop-down list that allows the user to select a different item.
- Additionally, you can allow the user to edit a selection.
- **ComboBox** inherits **ComboBoxBase**, which provides much of its functionality.
- Unlike **ListView**, which can allow multiple selections, **ComboBox** is designed for single selection.

ComboBoxes

- **ComboBox** is a generic class

  class ComboBox<T>

- Here, **T** specifies the type of entries.
- **ComboBox** defines two constructors.
  - The first is the default constructor
  - The second lets you specify the entries in the list.

    ComboBox(ObservableList<T> *list*)

ComboBoxes

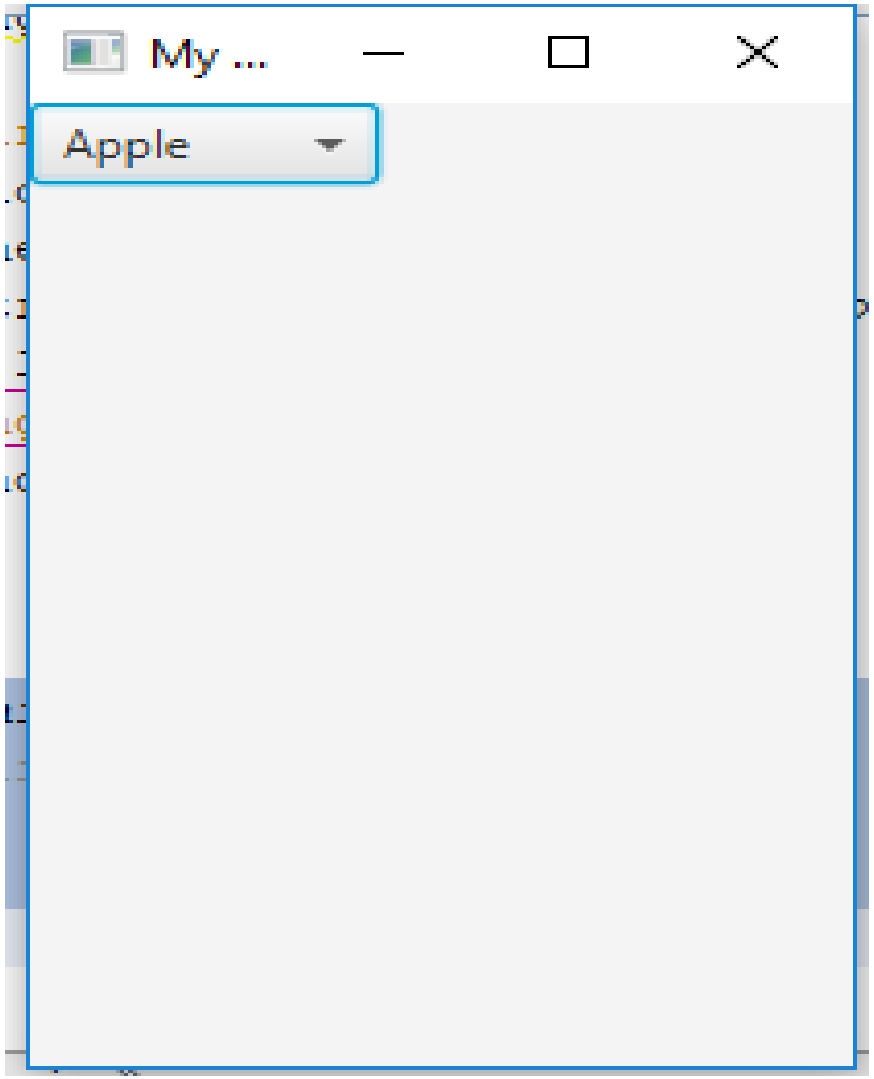
- You can obtain the current selection by calling **getValue( )**,
  - final T getValue( )
- To set the value of a **ComboBox** under program control, call **setValue( )**:
  - final void setValue(T *newVal*)

Example

```
import javafx.application.*;
import javafx.collections.*;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
public class Test extends Application
{
```

```
public void start(Stage s)
        {
        s.setTitle("My First Page");
        FlowPane f=new FlowPane();
        Scene s1=new Scene(f,200,300);
       ObservableList<String>
list=FXCollections.observableArrayList("Apple","Ma
ngo","Grapes","Banana","Guava");
        ComboBox<String> l1=new ComboBox(list);
       f.getChildren().addAll(l1);
      s.setScene(s1);
      s.show();
      }
```

```
public static void main(String[] args) {
launch(args);
}
}
```



ChoiceBox

- Another variation on the list control is **ChoiceBox. ChoiceBox** is somewhat like a **ComboBox** but without the editing capabilities. It lets the user select an option from a drop-down list.
- The selection is shown as checked in the list. Only single selection is supported.
- This makes **ChoiceBox** a useful alternative to radio buttons when space is an issue

ChoiceBox

- **ChoiceBox** is a generic class that is declared like this:

  class ChoiceBox<T>

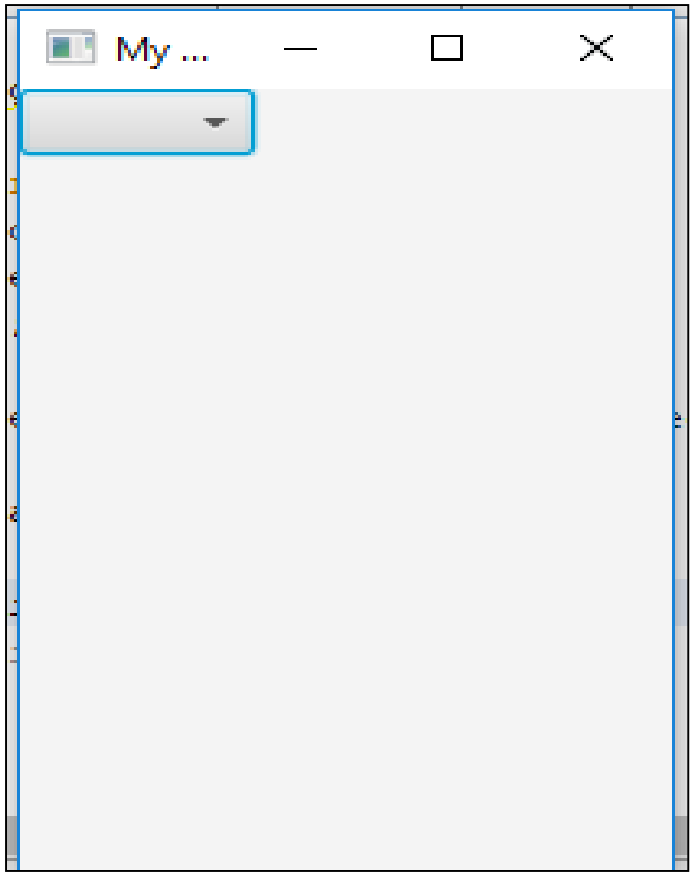
- **ChoiceBox** defines two constructors.
  - The first is the default constructor, which creates an empty **ChoiceBox**.
  - The second lets you specify the list of entries.

    ChoiceBox(ObservableList<T> *list*)

Example

```
ObservableList<String>
list=FXCollections.observableArrayList("Apple","
Mango","Grapes","Banana","Guava");
ChoiceBox<String> l1=new ChoiceBox(list);
 f.getChildren().add(l1);
```

Output



Adding Tooltips

- A tooltip is a short message that is displayed when the mouse hovers over a control.
- In JavaFX, a tooltip can easily be added to any control.
- To add a tooltip, you will call the **setTooltip( )** method defined by **Control**.
- The **Tooltip** class encapsulates a tooltip.
  Tooltip(String *str*)
- To set Tooltip

Tooltip t=new Tooltip(String);

control.setTooltip(t);

```
import javafx.application.*;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
public class Test extends Application{
public void start(Stage s)
            {
            s.setTitle("My First Page");
            FlowPane f=new FlowPane();
            Scene s1=new Scene(f,200,300);
           Button b=new Button("Username");
            Tooltip t=new Tooltip("press the button to submit the page");
            b.setTooltip(t);
       f.getChildren().add(b);
```

```
s.setScene(s1);
        s.show();
        }
    public static void main(String[] args) {
     launch(args);

    }


}
```

Layout Panes

- the layout of components on the screen is managed by a layout pane
- Types:
  - FlowPane
  - HBox
  - VBox
  - BorderPane
  - StackPane
  - GridPane

FlowPane

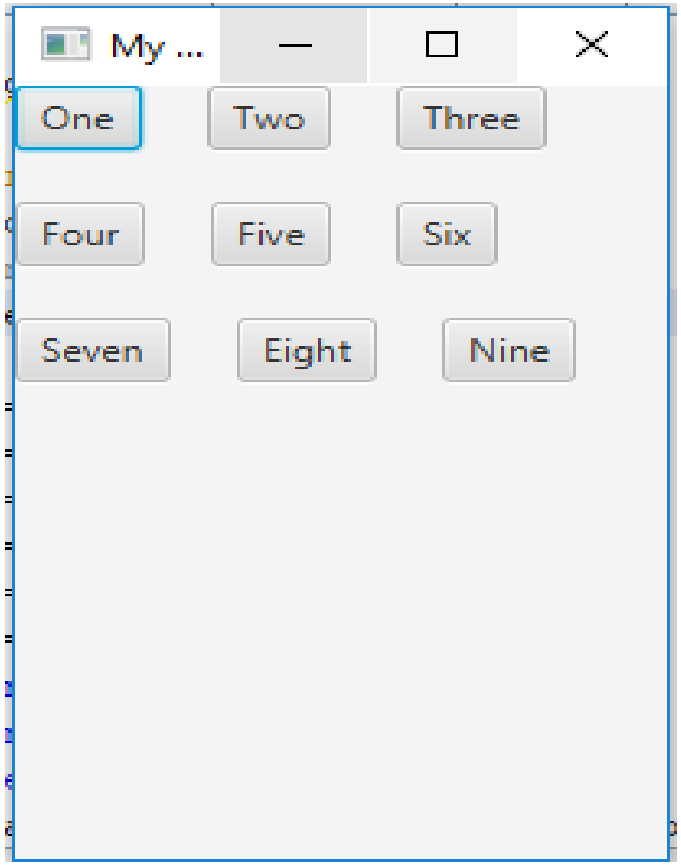
- **FlowPane** lays out content line by line, with lines wrapping as needed.
- By default, a horizontal flow is used, but it is possible to specify a vertical flow.
- **FlowPane(double h, double v)**: Creates a new Horizontal FlowPane layout, with specified horizontal and vertical gap.
- Set Orientation.VERTICAL

```
import javafx.application.*;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
public classTest extends Application{
public void start(Stage s)
        {
        s.setTitle("My First Page");
        FlowPane f=new FlowPane(20,20);
        Scene s1=new Scene(f,200,300);
         Button b1=new Button("One");
         Button b2=new Button("Two");
         Button b3=new Button("Three");
         Button b4=new Button("Four");
         Button b5=new Button("Five");
         Button b6=new Button("Six");
        Button b7=new Button("Seven");
```

```
        Button b8=new Button("Eight");
            Button b9=new Button("Nine");
        f.getChildren().addAll(b1,b2,b3,b4,b5,b6,b7,b8,b9);
         s.setScene(s1);
       s.show();
       }
    public static void main(String[] args) {
launch(args);

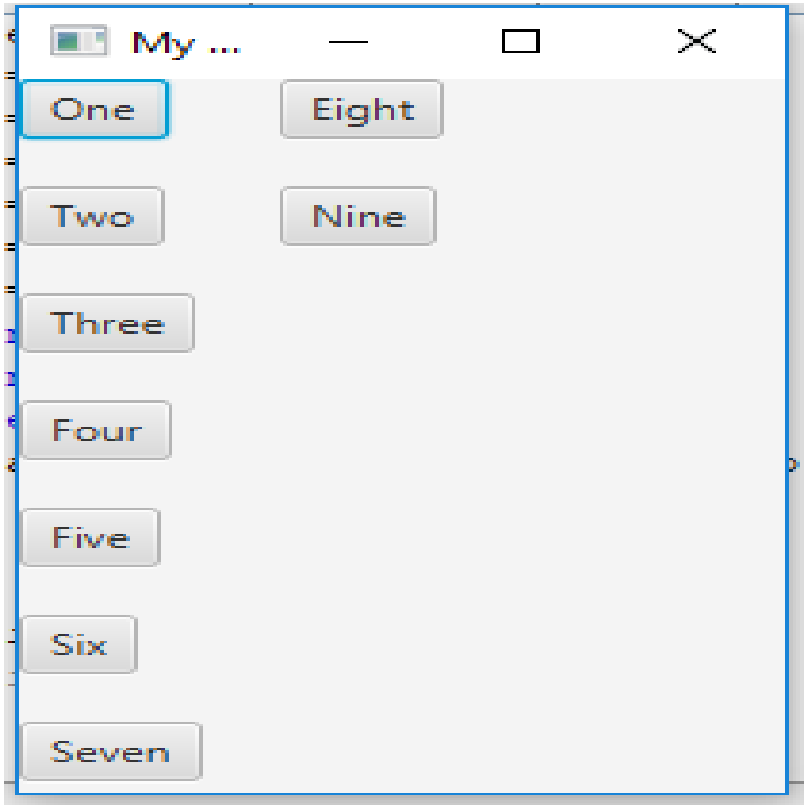
    }


}
```


My ...
One
Two
Three
Four
Five
Six
Seven
Eight
Nine

Vertical Orientation

- FlowPane f=new FlowPane(Orientation.VERTICAL,20,20);



HBox and VBox

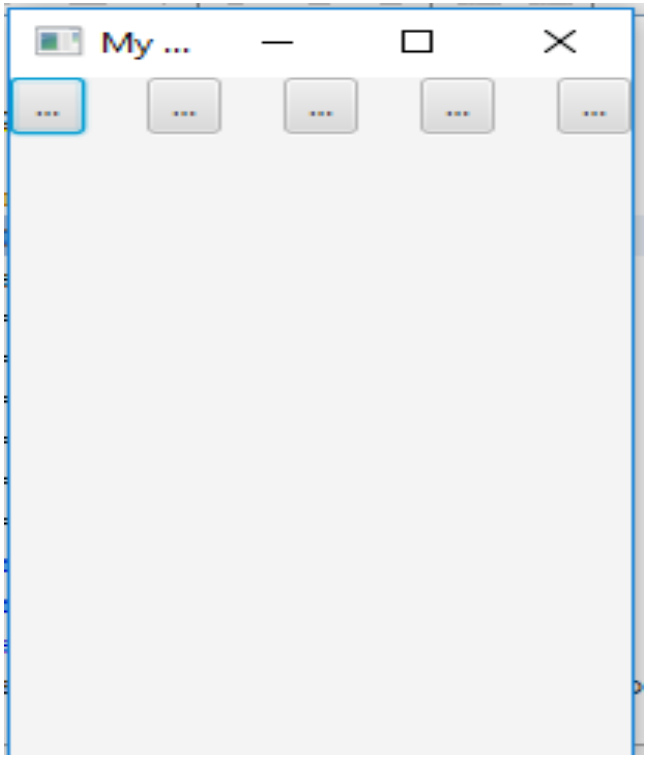
- Two simple but effective layout panes are **HBox** and **VBox.**
- **HBox** organizes its contents into a horizontal line.
- **VBox** lays out its contents in a vertical column.
- A principal advantage of **HBox** and **VBox** is that their contents will not be reorganized when the pane's size is changed.
- Thus, the contents will not reflow as they do in a **FlowPane**

HBox and VBox

- HBox(double *hGap*)
- VBox(double *vGap*)
- Here, *hGap* specifies the horizontal space between elements and *vGap* specifies the vertical gap.
- When the default constructor is used, the gap is zero in both cases.

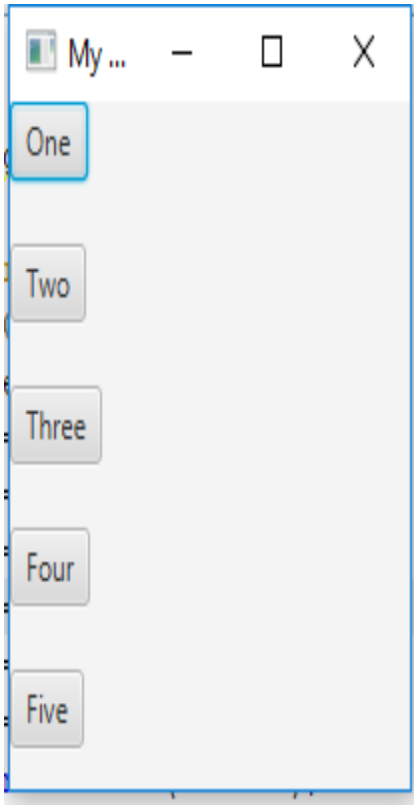
HBox-Example

HBox f=new HBox(20);



VBox-Example

VBox f=new VBox(20);



BorderPane

- **BorderPane** implements a layout style that defines five locations to which an item can be added.
- The first is the center.
- The other four are the sides (i.e., borders): top, bottom, left, and right. **BorderPane** is quite useful when you want to organize a window that has a header and footer, content, and various controls on the left and/or right.

BorderPane

- **BorderPane** defines three constructors.
- The first is the default constructor. When using the default constructor, you can assign a node to a location by use of the following methods:
- final void setCenter(Node *item*)
- final void setTop(Node *item*)
- final void setBottom(Node *item*)
- final void setLeft(Node *item*)
- final void setRight(Node *item*)

BorderPane

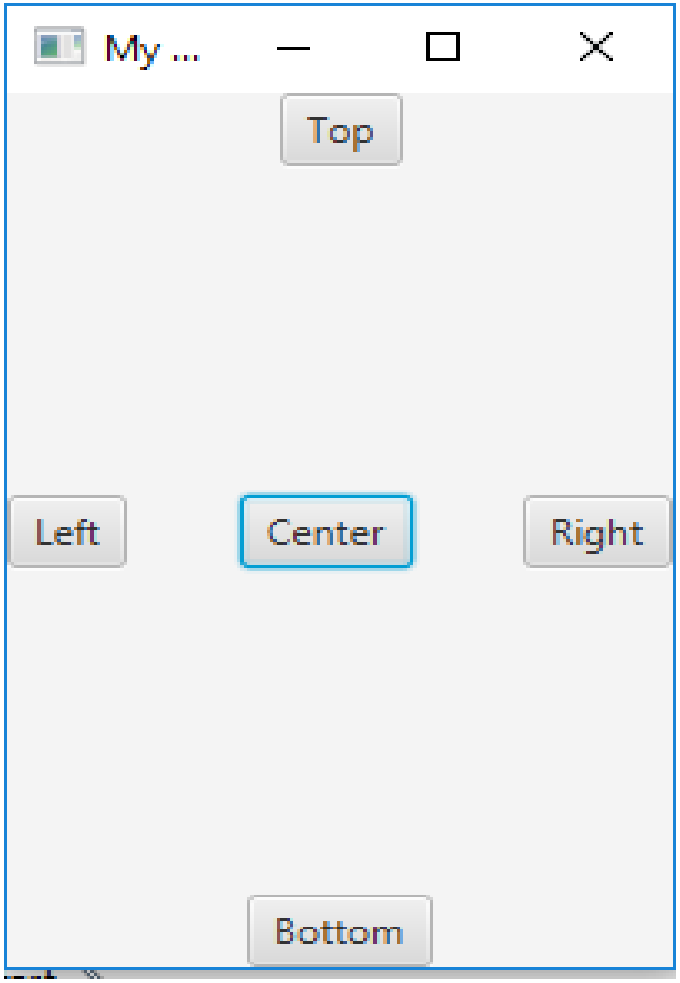
- The other two **BorderPane** constructors are shown here:
- BorderPane(Node *centerPos*)
- BorderPane(Node *centerPos*, Node *topPos*, Node *rightPos*, Node *bottomPos*, Node *leftPos*)
- You can set the alignment used in a location by calling **static void setAlignment(Node *what*, Pos *how*)**
- Here, *what* specifies the element being aligned and *how* specifies the alignment.
- **Pos** is an enumeration that specifies alignment constants, such as **Pos.CENTER, Pos.BOTTOM_RIGHT**, and **Pos.TOP_LEFT**.

```
import javafx.application.*;
import javafx.geometry.Pos;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
public class Test extends Application{
public void start(Stage s)
        {
        s.setTitle("My First Page");
        Button b1=new Button("Center");
         Button b2=new Button("Top");
         Button b3=new Button("Right");
         Button b4=new Button("Bottom");
         Button b5=new Button("Left");
         BorderPane f=new BorderPane(b1,b2,b3,b4,b5);
```

```
            BorderPane.setAlignment(b2, Pos.CENTER);
            BorderPane.setAlignment(b3, Pos.CENTER);
            BorderPane.setAlignment(b4, Pos.CENTER);
            BorderPane.setAlignment(b5, Pos.CENTER);
            Scene s1=new Scene(f,200,300);
            s.setScene(s1);
            s.show();
        }
    public static void main(String[] args)
 {
launch(args);

    }


}
```


My ...
Top
Left
Center
Right
Bottom

StackPane

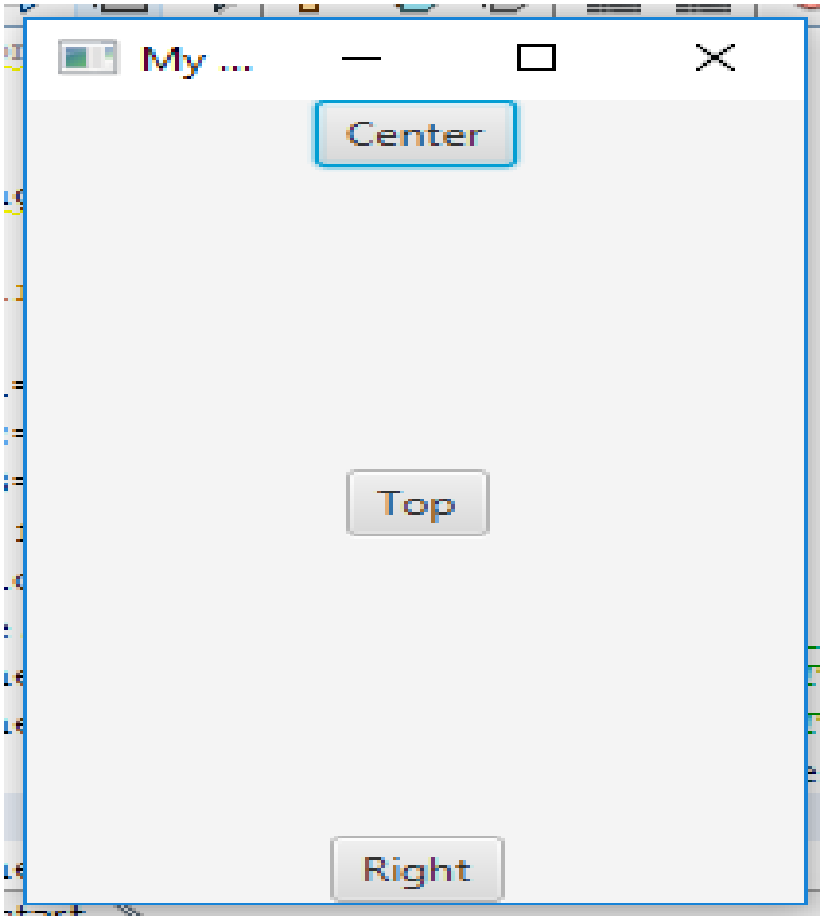
- layout is based on the Z-order, with one node placed on top of another, in a stack-like fashion.
- This layout can be very useful when you want to overlay one control with another.
- **StackPane** provides two constructors. The one we will use is the default constructor.
- The second, added by JavaFX 8, lets you specify a list of child nodes.
- You can set the alignment of each node in a **StackPane** by use of the static **setAlignment( )** method

```
import javafx.application.*;
import javafx.geometry.Pos;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
public class Test extends Application{
public void start(Stage s)
        {
        s.setTitle("My First Page");
        Button b1=new Button("Center");
        Button b2=new Button("Top");
        Button b3=new Button("Right");
       StackPane f=new StackPane();
        f.getChildren().addAll(b1,b2,b3);
        StackPane.setAlignment(b1, Pos.TOP_CENTER);
         StackPane.setAlignment(b2,Pos.CENTER);
         StackPane.setAlignment(b3, Pos.BOTTOM_CENTER);
```

```
            Scene s1=new Scene(f,200,300);
             s.setScene(s1);
             s.show();
        }
    public static void main(String[] args)
{
launch(args);

    }

}
```


My ...
Center
Top
Right

GridPane

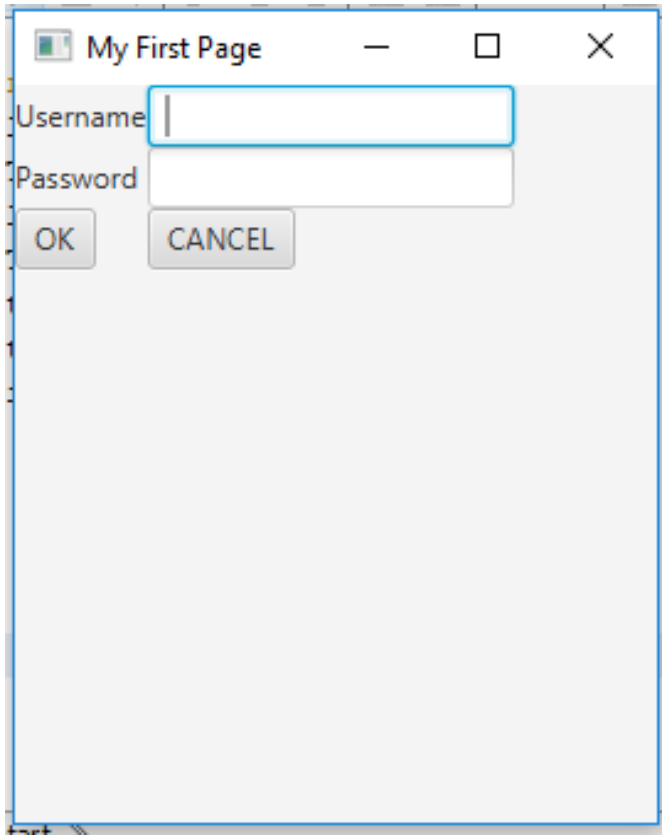
- Used to lay out controls using a row/column format
- **GridPane** provides only the default constructor.
- The location at which a child node is added to the grid is specified by setting its row and column indices.
- static void setConstraints(Node *what*, int *column*, int *row*)
- **setRowIndex( )** and **setColumnIndex( )**
- void add(Node *child*, int *column*, int *row*)

```
import javafx.application.*;
import javafx.geometry.Pos;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
public class Test extends Application{
public void start(Stage s)
        {
        s.setTitle("My First Page");
        Label l1=new Label("Username");
    TextField t1=new TextField();
    Label l2=new Label("Password");
    TextField t2=new TextField();
    Button b1=new Button("OK");
    Button b2=new Button("CANCEL");
    GridPane g=new GridPane();
```

```
g.add(l1, 0, 0);
        g.add(t1, 1, 0);
        g.add(l2,0,1);
        g.add(t2,1,1);
        g.add(b1,0,2);
        g.add(b2,1,2);
Scene s1=new Scene(g,200,300);
              s.setScene(s1);
              s.show();
       }
   public static void main(String[] args)
{
launch(args);

   }


}
```


My First Page
Username
Password
OK
CANCEL

Event Handling

- In JavaFX, an *event* is an object that describes some action that has affected the program, such as when the user interacts with the program by clicking a button.

Event Basics

- JavaFX uses what is, in essence, the *delegation event model* approach to event handling.
- The concept behind this model is quite simple:
- a source, such as a control, generates an event and sends it to one or more listeners, which handle the event.
- To receive an event, the handler for the event must first be registered with the event source.
- When the event occurs, the handler is called.
- It must then respond to the event and return.
- Thus, a user-interface element *delegates* the processing of an event to a separate event handler.

The Event Class

- The base class for JavaFX events is the **Event** class, which is packaged in **javafx.event.**
- **Event** inherits **java.util.EventObject**, which means that JavaFX events share the same basic functionality as other Java events.
- When a JavaFX event is generated, it is encapsulated within an **Event** instance.
- **Event** supports several methods that help you manage events. For example, you can obtain the source of the event and the event type.
- Several subclasses of **Event** are defined, which represent various types of events.
- The one that we will use to introduce event handling is **ActionEvent**.
- It encapsulates action events generated by several JavaFX controls, including the push button.

The EventHandler Interface

- Events are handled by implementing the **EventHandler** interface, which is also in **javafx.event**.
- It is a generic interface with the following form:

Interface EventHandler<T extends Event>

- Here, **T** specifies the type of event that the handler will handle.
- It defines one method, called **handle( )**, which receives the event object as an argument.
- It is shown here:

  void handle(T *eventObj*)

- Here, *eventObj* is the event that was generated.

Two basic ways to specify a handler for an event

- **First, you can call addEventHandler( ), which is defined by the Node class, among others.**
- **Second, in many cases, you can use what is referred to as a *convenience method*.**
  - Convenience methods use the prefix **setOn** and set an event handler property.
- In general, if a convenience method exists for the event that you want to handle, it is the recommended approach, and is the approach used here.
- With either approach, when an event is generated, its handler is called on the program's application thread.
- One other point: if you want to unregister an event handler, call **removeEventHandler( )**.

Example

```
import javafx.application.*;
import javafx.event.EventHandler;
import javafx.event.ActionEvent;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
```

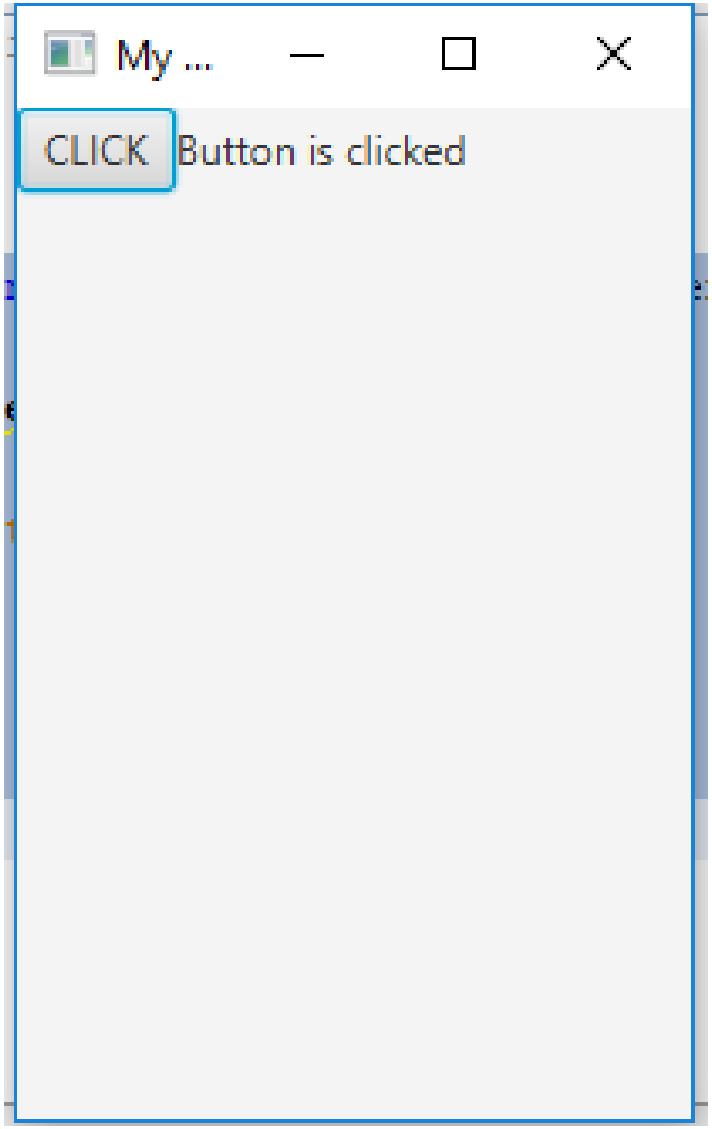
Example

```
public class Test extends Application{
public void start(Stage s)
        {
        Label l=new Label();
        s.setTitle("My First Page");
        FlowPane f=new FlowPane();
        Scene s1=new Scene(f,200,300);
      Button b=new Button("CLICK");
      f.getChildren().addAll(b,l);
      Handler h=new Handler();
      b.setOnAction(h);
      s.setScene(s1);
      s.show();
      }
```

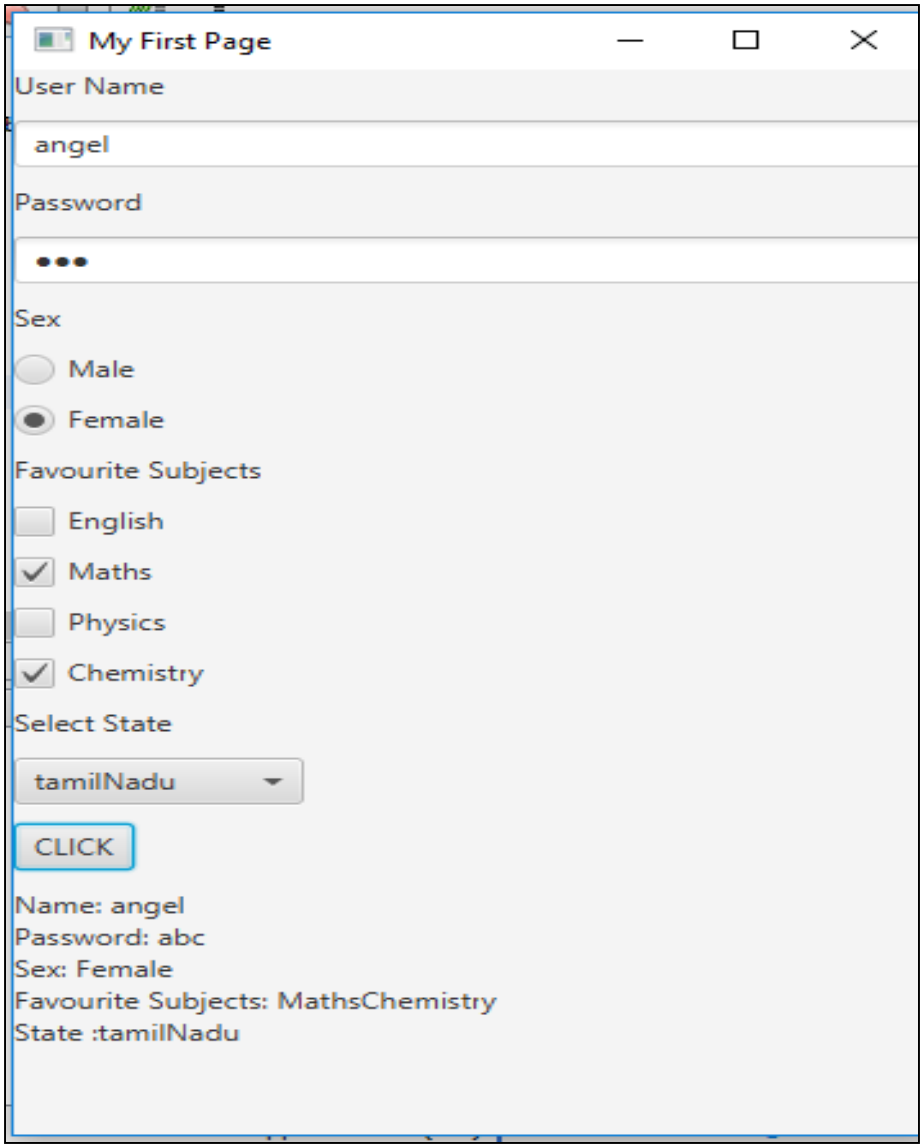
Example

```
public static void main(String[] args) {
launch(args);
}
 class Handler implements
EventHandler<ActionEvent>
    {
        public void handle(ActionEvent e)
        {
            l.setText("Button is clicked");
        }
}
}
```

OUTPUT



Handling Action Events



Handling Action Events

```
import javafx.application.*;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
```

```
public class Test extends Application{
	Label l=new Label("");
	Label l1=new Label("User Name");
	TextField t1=new TextField();
	Label l2=new Label("Password");
	PasswordField t2=new PasswordField();
	Label l3=new Label("Sex");
	RadioButton r1=new RadioButton("Male");
	RadioButton r2=new RadioButton("Female");
	Label l4=new Label("Favourite Subjects");
	CheckBox c1=new CheckBox("English");
	CheckBox c2=new CheckBox("Maths");
	CheckBox c3=new CheckBox("Physics");
	CheckBox c4=new CheckBox("Chemistry");
	Label l5=new Label("Select State");
	ObservableList<String>
list=FXCollections.observableArrayList("tamilNadu","Kerala","Karnataks","Guj
arat","Maharashtra");
	ComboBox<String> ls1=new ComboBox(list);
```

```
public void start(Stage s)
        {
        s.setTitle("My First Page");
 VBox f=new VBox(10);
ToggleGroup t=new ToggleGroup();
 r1.setToggleGroup(t);
 r2.setToggleGroup(t);
        Button b=new Button("CLICK");
 Handler h=new Handler();
 b.setOnAction(h);
Scene s1=new Scene(f,200,300);
 f.getChildren().addAll(l1,t1,l2,t2,l3,r1,r2,l4,c1,c2,c3,c4,l5,ls1,b,l);
         s.setScene(s1);
        s.show();
 }
```

```
public static void main(String[] args)
  {
     launch(args);

  }
  class Handler implements EventHandler<ActionEvent>
  {
     public void handle(ActionEvent e)
     {
        String s1=t1.getText();
        String s2=t2.getText();
        String s3="";
        if(r1.isSelected()==true)
           s3+=r1.getText();
        else
           s3+=r2.getText();
```

```
String s4="";
        if(c1.isSelected()==true)
           s4+=c1.getText();
         if(c2.isSelected()==true)
           s4+=c2.getText();
          if(c3.isSelected()==true)
           s4+=c3.getText();
           if(c4.isSelected()==true)
           s4+=c4.getText();
           String s5=ls1.getValue();
        l.setText("Name: "+s1+"\n"+"Password:
"+s2+"\n"+"Sex: "+s3+"\n"+"Favourite Subjects:
"+s4+"\n"+"State :"+s5);
}   }
}
```

Key Events

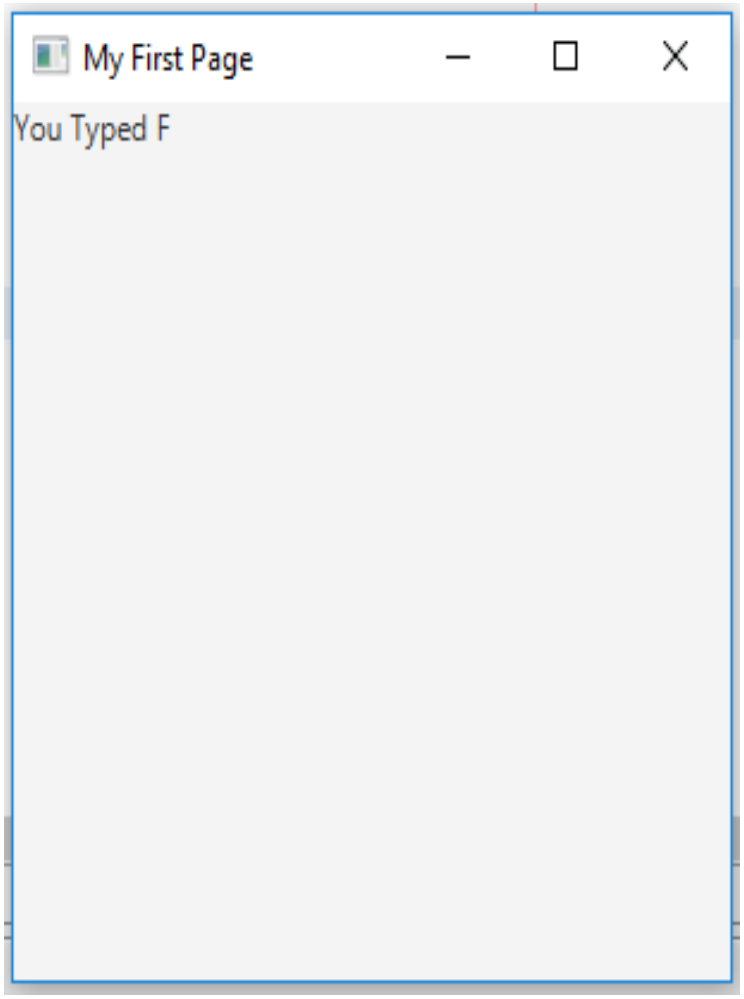
- When a key is typed on the keyboard, a **KeyEvent** is generated.
- There are three types of key events: a key is pressed; a key is released; and a key is typed. These events are represented by the fields **KEY_PRESSED, KEY_RELEASED**, and **KEY_TYPED**, which are defined by **KeyEvent** as objects of **EventType**.
- A key-pressed event is generated when a key is pressed on the keyboard.
- A key-released event is generated when the key is released.
- A key-typed event is generated when a normal character on the keyboard, such as A, **K**, 9, or + is typed.

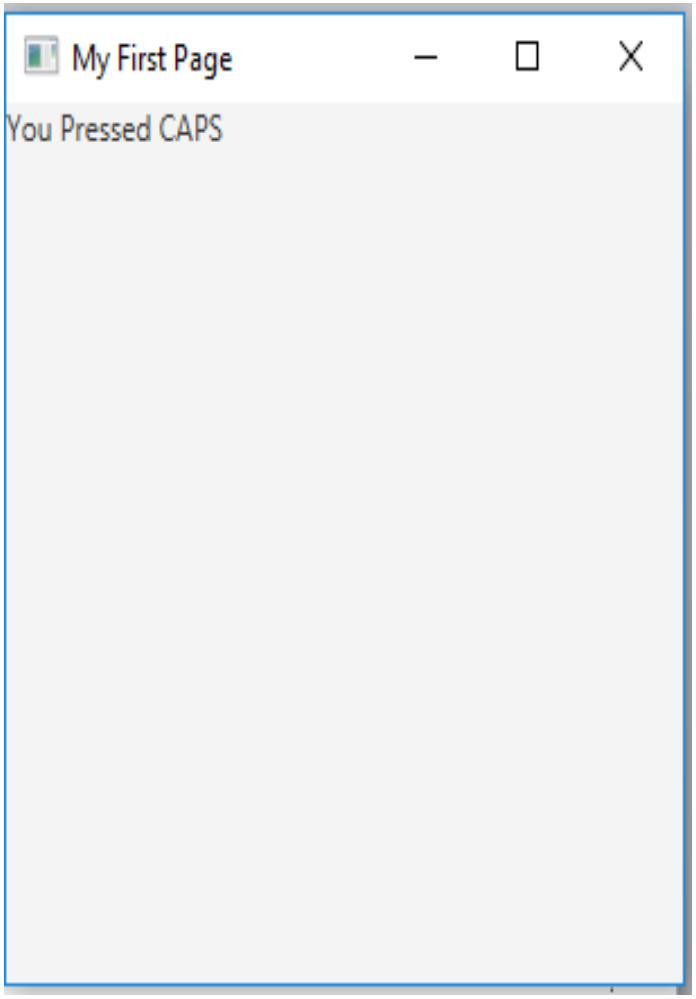
Convenience methods

- Convenience methods that make it easy to register an event handler for the various types of key events.
  - final void setOnKeyPressed(EventHandler<? super KeyEvent> *handler*)
  - final void setOnKeyReleased(EventHandler<? super KeyEvent> *handler*)
  - final void setOnKeyTyped(EventHandler<? super KeyEvent> *handler*)
- When a key-typed event is received, you can obtain the character by calling **getCharacter( )** on the event

- For key-pressed and key-released events, you can obtain the key code associated with the event. You can obtain the key code by calling **getCode( )** on the key event.

Key Code	Meaning
RIGHT	The stand-alone RIGHT ARROW key
LEFT	The stand-alone LEFT ARROW key
KP_RIGHT	The RIGHT ARROW key on the number pad
KP_LEFT	The LEFT ARROW key on the number pad
F1	The F1 key
F10	The F10 key
ALT	The ALT key
CONTROL	The CTRL key
SHIFT	The SHIFT key


My First Page
You Typed F


My First Page
You Pressed CAPS

```
import javafx.application.*;
import javafx.event.EventHandler;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
import javafx.scene.input.KeyEvent;
public class Test extends Application{
        Label l=new Label("");
```

```
public void start(Stage s)
        {
        s.setTitle("My First Page");
    FlowPane f=new FlowPane();
    Scene s1=new Scene(f,300,300);
    Handler1 h1=new Handler1();
    s1.setOnKeyTyped(h1);
    Handler2 h2=new Handler2();
    s1.setOnKeyPressed(h2);
    f.getChildren().add(l);
        s.setScene(s1);
        s.show();
    }
  public static void main(String[] args)
  {
    launch(args);

  }
```

```
class Handler1 implements EventHandler<KeyEvent>
   {
      public void handle(KeyEvent e)
      {
         l.setText("You Typed "+e.getCharacter());
      }
   }
class Handler2 implements EventHandler<KeyEvent>
   {
      public void handle(KeyEvent e)
      {
         l.setText("You Pressed "+e.getCode());
      }
   }
}
```

Mouse Events

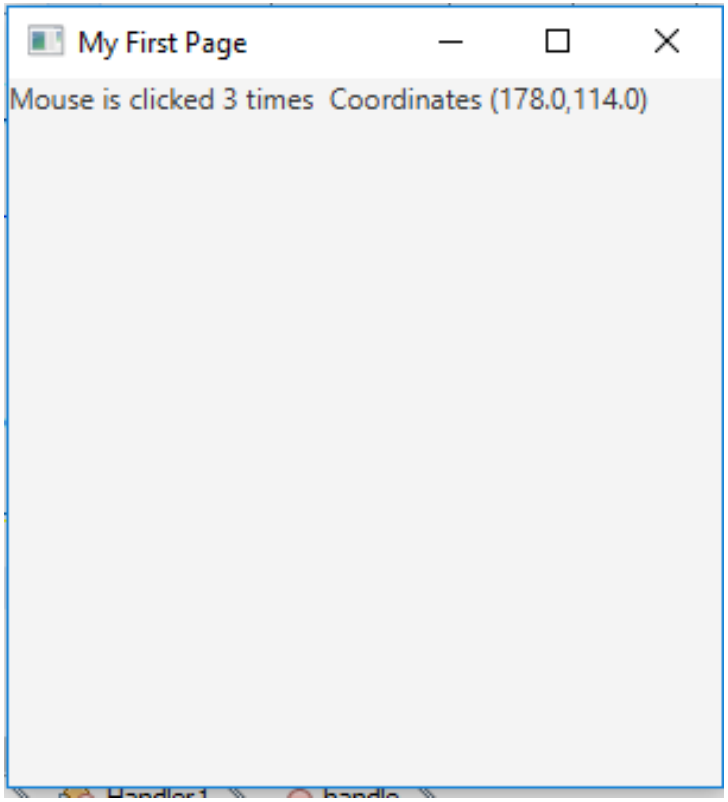
- Mouse events are represented by the **MouseEvent** class
- There are a number of different types of events that can be generated by the mouse.
- For example, an event is generated when the mouse is moved, when a button is clicked or released, when the mouse enters or exits an element that handles mouse events, or when the mouse is dragged.
- As a result, a number of **EventType** objects are defined by **MouseEvent** that represent the events, such as **MOUSE_CLICKED** and **MOUSE_MOVED**.

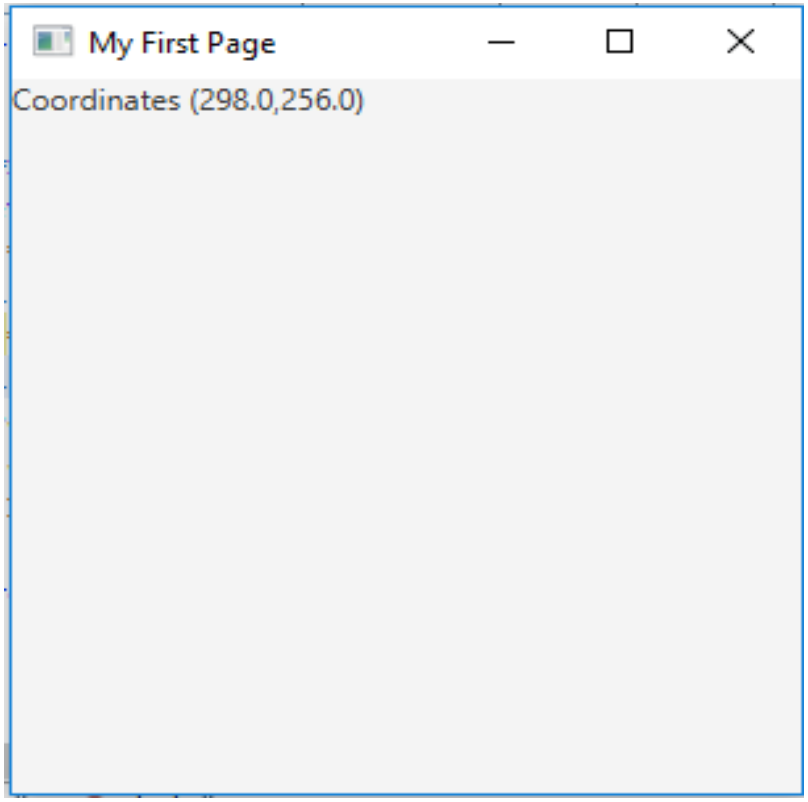
Mouse Events

- convenience methods that can be used to register event handlers for mouse events. The two used in the example that follows are shown here:
  - final void setOnMouseClicked(EventHandler<? super MouseEvent> *handler*)
  - final void setOnMouseMoved(EventHandler<? super MouseEvent> *handler*)
- When the mouse is clicked, you can find out which button was used by calling **getButton( )**

Mouse Events

- In general, if the left button was clicked, **MouseButton.PRIMARY** is returned.
- If the right button was clicked, the return value is **MouseButton.SECONDARY**.
- For a mouse with a middle button, clicking it causes **MouseButton.MIDDLE** to be returned.
- If no button was clicked, such as in the case of **MouseEvent** resulting from a move, then **getButton( )** returns **MouseButton.NONE**
- You can obtain the number of times a mouse button has been clicked by calling **getClickCount( )**
- You can obtain the location of the mouse at the time an event occurred by **getSceneX( )** and **getSceneY( ).**


My First Page
Mouse is clicked 3 times Coordinates (178.0,114.0)


My First Page
Coordinates (298.0,256.0)

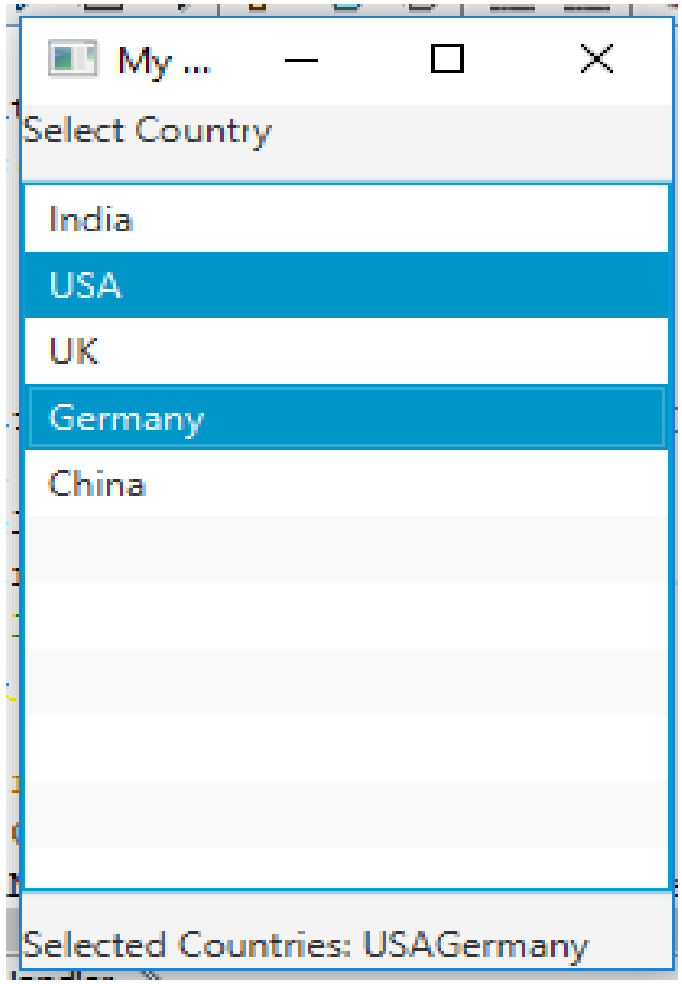
Example

```
import javafx.application.*;
import javafx.event.EventHandler;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
import javafx.scene.input.MouseEvent;
public class JavaApplication15 extends Application{
        Label l=new Label("");
```

```
public void start(Stage s)
        {
        s.setTitle("My First Page");
      FlowPane f=new FlowPane();
      Scene s1=new Scene(f,300,300);
      Handler1 h1=new Handler1();
      s1.setOnMouseClicked(h1);
      Handler2 h2=new Handler2();
      s1.setOnMouseMoved(h2);
      f.getChildren().add(l);
            s.setScene(s1);
            s.show();
      }
```

```
public static void main(String[] args)
   {
      launch(args);
}
   class Handler1 implements EventHandler<MouseEvent>
   {
      public void handle(MouseEvent e)
      {
         l.setText("Mouse is clicked "+e.getClickCount()+ " times"+ "
Coordinates ("+e.getSceneX()+","+e.getSceneY()+")");
      }
   }
     class Handler2 implements EventHandler<MouseEvent>
   {
      public void handle(MouseEvent e)
      {
         l.setText("Coordinates ("+e.getSceneX()+","+e.getSceneY()+")");
      }
   }
}
```

Handling Change Events



```
import javafx.application.*;
import javafx.beans.value.ObservableValue;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
import javafx.beans.value.ChangeListener;
```

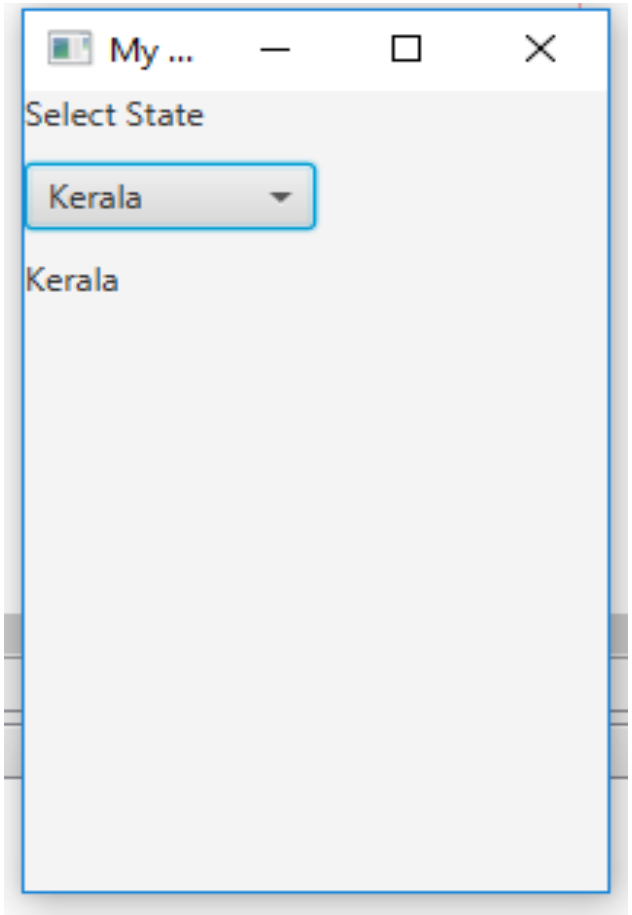
```
public class Test extends Application{
	Label l=new Label("");
	Label l1=new Label("Select Country");
	ObservableList<String>
list1=FXCollections.observableArrayList("India","USA","UK","G
ermany","China");
	ListView<String> ls1=new ListView(list1);
	public void start(Stage s)
	    {
	    s.setTitle("My First Page");
	VBox f=new VBox(10);
	    MultipleSelectionModel<String>
m1=ls1.getSelectionModel();
	m1.setSelectionMode(SelectionMode.MULTIPLE);
      Handler h=new Handler();
```

```
m1.selectedItemProperty().addListener(h);
    Scene s1=new Scene(f,200,300);
     f.getChildren().addAll(l1,ls1,l);
           s.setScene(s1);
           s.show();
    }
  public static void main(String[] args)
  {
     launch(args);

  }
```

```
class Handler implements ChangeListener<String>
   {
       public void changed(ObservableValue<? extends String> e, String
oldValue, String newValue)

  {
         String t="";
          ObservableList<String>
s=ls1.getSelectionModel().getSelectedItems();
          for(String s1:s)
             t+=s1;
         l.setText("Selected Countries: "+t);
       }
}
}
```


My ...
Select State
Kerala
Kerala

```
import javafx.application.*;
import javafx.beans.value.ObservableValue;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
import javafx.beans.value.ChangeListener;
```

```
public class JavaApplication15 extends Application{
    Label l=new Label("");
        Label l2=new Label("Select State");
        ObservableList<String>
list2=FXCollections.observableArrayList("tamilNadu","Kerala","Karna
taka","Gujarat","Maharashtra");
    ChoiceBox<String> ls2=new ChoiceBox(list2);
    public void start(Stage s)
        {
        s.setTitle("My First Page");
    VBox f=new VBox(10);
            Handler1 h1=new Handler1();
   SingleSelectionModel<String> m2=ls2.getSelectionModel();
   m2.selectedItemProperty().addListener(h1);
  Scene s1=new Scene(f,200,300);
   f.getChildren().addAll(l2,ls2,l);
```

```
s.setScene(s1);
            s.show();
      }
   public static void main(String[] args)
   {
      launch(args);
}
class Handler1 implements ChangeListener<String>
   {
      public void changed(ObservableValue<? extends String> e, String
oldValue, String newValue)

      {
String s=ls2.getSelectionModel().getSelectedItem();
         l.setText(s);
      }
}   }
```

The Event Dispatch Chain

- In JavaFX, events are processed via an *event dispatch chain*.
- As it relates to a GUI, the event dispatch chain is a path from the top element in the scene graph (typically the stage) to the target of the event, which is the control that generated the event.
- When an event is processed, two main phases occur.
  - First, the event is passed from the top element down the chain to the target of the event. This is called *event capturing*.
  - After the target node processes the event, the event is passed back up the chain, thus allowing parent nodes a chance to process the event, if required. This is called *event bubbling*. Event handlers are called during the event bubbling phase.

The Event Dispatch Chain

- It is also possible for an application to implement an *event filter*.
- An event filter is, in essence, a special type of event handler.
- Event filters are called during the event capturing phase.
- Thus, an event filter for an event will execute before an event handler for the same event.
- It is also possible for a node in the chain to *consume* an event, which prevents it from being processed further.
- Therefore, if a filter consumes an event, an event handler for that event will not execute.
- In general, event filters constitute a special case and are not often needed, but when they *are* needed, they can be very useful.

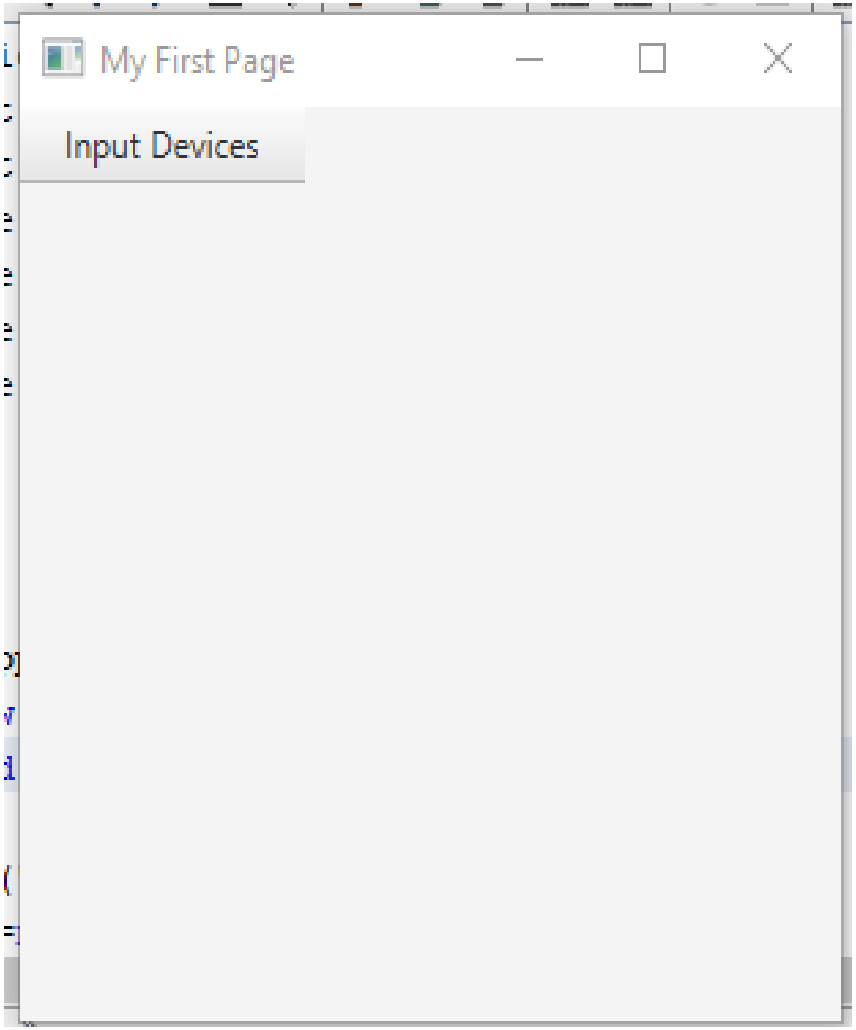
Explore Menus

- The JavaFX menu system supports several key elements, including
  - The menu bar, which is the main menu for an application.
  - The standard menu, which can contain either items to be selected or other menus (submenus).
  - The context menu, which is often activated by right-clicking the mouse. Context menus are also
  - called popup menus.
  - Several different types of menu items, including the ability to create custom menu items.
  - Accelerator keys, which enable menu items to be selected without having to activate the menu.
  - Mnemonics, which allow a menu item to be selected by the keyboard once the menu options are
  - displayed.

Menu Basics

- To provide a main menu for an application, you need an instance of **MenuBar**, a container for menus.
- The **MenuBar** contains instances of **Menu**. Each **Menu** object defines a menu.
- That is, each **Menu** object contains one or more selectable items.
- The items displayed by a **Menu** are objects of type **MenuItem**.
- Thus, a **MenuItem** defines a selection that can be chosen by the user.

Class	Description
CheckMenuItem	A check menu item.
ContextMenu	A popup menu that is typically activated by right-clicking the mouse.
CustomMenuItem	A menu item that can contain any type of **Node**.
Menu	A standard menu. A menu consists of one or more **MenuItem**s.
MenuBar	An object that holds the top-level menu for the application.
MenuButton	A control that actives a drop-down menu when pressed.
MenuItem	An object that populates menus.
RadioMenuItem	A radio menu item.
SeparatorMenuItem	Provides a visual separator between menu items.
SplitMenuButton	A variation of **MenuButton** that provides both a button and a drop-down menu activator in one control.


My First Page
Input Devices

```
import javafx.application.*;
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.scene.*;
import javafx.scene.control.*;
import javafx.stage.*;
import javafx.scene.layout.*;
public class Test extends Application{
	Label l=new Label("");
	public void start(Stage s)
		{
		s.setTitle("My First Page");
	FlowPane f=new FlowPane();
	Scene s1=new Scene(f,300,300);
	MenuBar m=new MenuBar();
	Menu m1=new Menu("Input Devices");
	MenuItem m11=new MenuItem("Keyboard");
	 MenuItem m12=new MenuItem("Mouse");
```

```
m1.getItems().addAll(m11,m12);
        m.getMenus().add(m1);
       f.getChildren().addAll(m,l);
       Handler h=new Handler();
      m11.setOnAction(h);
      m12.setOnAction(h);
             s.setScene(s1);
             s.show();
       }
   public static void main(String[] args)
   {
      launch(args);
     }
```

```
class Handler implements
EventHandler<ActionEvent>
   {
      public void handle(ActionEvent e)
      {
         String
name=((MenuItem)e.getTarget()).getText();
         l.setText(name +" is selected");
      }
   }
}
```